Edge AI Systems: Where Should the Model Run?

Class 20 · CIS 4930/5930 Future Edge Networks and Distributed Intelligence

Xin Liu

Department of Computer Science, Florida State University

Monday, November 2, 2026

One request, three places it could run

"Summarize the 20-minute voicemail from my doctor."

≈4K-token prompt · ≈200-token answer · private



Phone	Edge server	Cloud
+ no network, no queue	+ short RTT, real GPUs	+ largest models
+ data never leaves	+ bigger model	+ elastic capacity
– small model (1–4B)	– shared with other users	– longest RTT, queueing
– battery and heat	– only where deployed	– paid per token; data leaves

The same request can be served at all three. Which one is right depends on the request, the device, and the network. Today we make that choice precise.

RTT values are typical, illustrative figures; they vary widely by operator and deployment.

Why push models onto the device?

- **Latency and availability**: no round trip, no queue behind other users; works in a tunnel or on a plane.
- **Privacy**: messages, photos, and health data never leave the phone.
- **Cost**: every cloud token burns data-center GPU time; on-device tokens run on hardware the user already paid for.
- **Personal context**: the phone already holds your calendar, mail, and screen.

Shipping and measured

- **Apple Intelligence**: a ∼3B on-device model, plus a larger server model on Private Cloud Compute. This is already a router: on-device first, server when needed.
- **MobileLLM** (ICML'24): sub-billion models designed for phones. Today's paper, MobileMoE, continues this line.
- **MELTing Point** (MobiCom'24): on phones, LLM inference is largely *memory-bound*; energy and heat limit continuous use.

Apple, Apple Intelligence Foundation Language Models, arXiv:2407.21075, 2024 · Z. Liu et al., MobileLLM, ICML 2024 · S. Laskaridis et al., MELTing Point, ACM MobiCom 2024

Today's question, made precise

For a request r, choose a place $p \in \{\text{device}, \text{edge}, \text{cloud}\}$, a model m, and a precision b:

$$T(r,p) = \underbrace{T_{\text{up}} + T_{\text{down}}}_{\text{network}} + \underbrace{T_{\text{queue}}}_{\text{sharing}} + \underbrace{T_{\text{prefill}}(n_{\text{in}})}_{\text{read the prompt}} + \underbrace{n_{\text{out}} \cdot T_{\text{decode}}}_{\text{write the answer}}$$

subject to

- memory: weights + KV cache $\leq M_{\text{device}}$
- energy: within battery and thermal budget
- quality: $Q(m,b) \geq Q_{\min}$
- privacy: sensitive $r \Rightarrow$ trusted p

Each part of today attacks one term

1	The device's budget	what bounds T_{decode} and M
2	Quantization	fewer bytes per weight
3	Mixture of Experts	fewer weights read per token
4	KV cache	memory that grows with n
5	Split & collaborative	the network terms
6	Model routing	choose p and m per request

Roadmap

1. The device's budget: memory, bandwidth, energy
2. Quantization: fewer bits per weight
3. Mixture of Experts: capacity without compute
4. The KV cache: memory that grows with the conversation
5. Split and collaborative inference
6. Model routing: choosing the model per request
7. Wrap-up

Budget #1 — Memory: will the weights even fit?

$$\text{bytes}_{\text{weights}} = N_{\text{params}} \times \frac{b}{8}$$

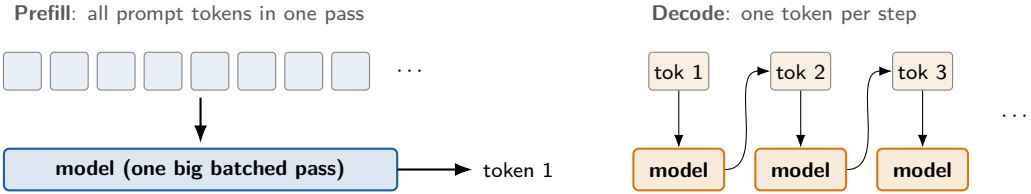
Params	FP16	INT8	INT4
1B	2 GB	1 GB	0.5 GB
3B	6 GB	3 GB	1.5 GB
8B	16 GB	8 GB	4 GB
70B	140 GB	70 GB	35 GB

- Flagship phone DRAM: 4 GB (iPhone 13) $\rightarrow$ 12 GB (iPhone 17); 12–16 GB (Galaxy S25 / S25 Ultra).
- That DRAM is *shared* by the OS, camera, GPU, and every open app. One app realistically gets a few GB.
- A data-center GPU (H100) has 80 GB of HBM, and a server has 8 of them.

Realistic on-device range today: about **1–4B parameters at 4 bits**.

Phone DRAM figures as reported in Y. Chen et al., MobileMoE: Scaling On-Device Mixture of Experts, arXiv:2605.27358, 2026

Two phases of every LLM request



Prefill

Matrix × matrix: each weight is reused by every prompt token.
⇒ **compute-bound**. Metric: time to first token (TTFT).

Decode

Matrix × vector: *every* weight is read again for *each* token.
⇒ **memory-bandwidth-bound**. Metric: time per output token (TPOT).

They differ so much that DistServe (OSDI'24) serves them on separate GPUs, and llm.npu (ASPLOS'25) moves phone prefill to the NPU, at over 1,000 tokens/s for a billion-sized model.

Y. Zhong et al., DistServe, USENIX OSDI 2024 · D. Xu et al., Fast On-device LLM Inference with NPUs, ACM ASPLOS 2025

Budget #2 — Bandwidth: decode speed is set by bytes, not FLOPs

On a phone the batch size is 1, so every new token reads every weight once:

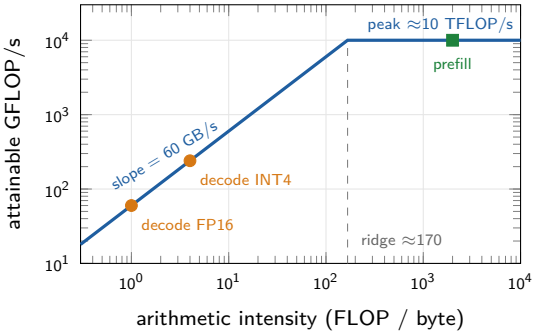
$$\text{tokens/s} \leq \frac{\text{memory bandwidth}}{\text{bytes read per token}} \approx \frac{BW}{\text{bytes}_{\text{weights}}}$$

Device	Bandwidth	Model	Bytes / token	Upper bound
Phone (LPDDR5X)	≈60 GB/s	3B, FP16	6 GB	≈10 tok/s
Phone	≈60 GB/s	3B, INT4	1.5 GB	≈40 tok/s
Phone	≈60 GB/s	8B, INT4	4 GB	≈15 tok/s
H100 GPU (HBM3)	3.35 TB/s	8B, FP16	16 GB	≈200 tok/s

Halve the bytes and you roughly double decode speed.
Quantization is a **speed** technique, not only a memory technique.

Upper bounds ignore KV-cache reads and runtime overhead. A server also batches many users, so one read of each weight serves many tokens.

The roofline view: where each phase sits



Arithmetic intensity I = FLOPs per byte moved.

- Decode: 2 FLOPs per weight over $b/8$ bytes per weight $\Rightarrow I = 16/b$ (FP16: 1; INT4: 4).
- Prefill over n tokens reuses each weight n times: $I \approx 16n/b$.
- Attainable $= \min(\text{peak}, BW \times I)$.

Decode uses under 3% of peak compute.
Fewer bytes help; faster math does not.

S. Williams, A. Waterman, D. Patterson, Roofline: An Insightful Visual Performance Model for Multicore Architectures, CACM 2009. Phone numbers are round, illustrative values.

Budget #3 — Energy: moving data costs more than math



- One DRAM read costs ≈170× a 32-bit float multiply, so **joules follow bytes moved**.
- A phone battery holds ≈15–20 Wh, and the phone can sustain only a few watts before it throttles.
- Measured on phones: "continuous execution of LLMs remains elusive" because of energy and heat (MELTing Point).

Fewer bytes per token buys speed *and* battery life.

M. Horowitz, Computing's Energy Problem (and What We Can Do About It), IEEE ISSCC 2014 · S. Laskaridis et al., MELTing Point, ACM MobiCom 2024

Quick check (2 minutes)

A phone has **12 GB** of DRAM and $\approx$**60 GB/s** of memory bandwidth.
You want a **7B** chat model at $\geq$**20 tokens/s**.

1. In FP16?
2. In INT4?
3. What would you change?

Quick check (2 minutes)

A phone has **12 GB** of DRAM and $\approx$**60 GB/s** of memory bandwidth.
You want a **7B** chat model at $\geq$**20 tokens/s**.

1. In FP16?
   $7 \times 2 = 14$ GB $>$ 12 GB: it does not fit. Even if it did, $60/14 \approx 4$ tok/s.
2. In INT4?
3. What would you change?

Quick check (2 minutes)

A phone has **12 GB** of DRAM and $\approx$**60 GB/s** of memory bandwidth.
You want a **7B** chat model at $\geq$**20 tokens/s**.

1. In FP16?
   $7 \times 2 = 14$ GB $>$ 12 GB: it does not fit. Even if it did, $60/14 \approx 4$ tok/s.
2. In INT4?
   3.5 GB fits, but decode is at most $60/3.5 \approx 17$ tok/s, still short of 20.
3. What would you change?

Quick check (2 minutes)

A phone has **12 GB** of DRAM and $\approx$**60 GB/s** of memory bandwidth.
You want a **7B** chat model at $\geq$**20 tokens/s**.

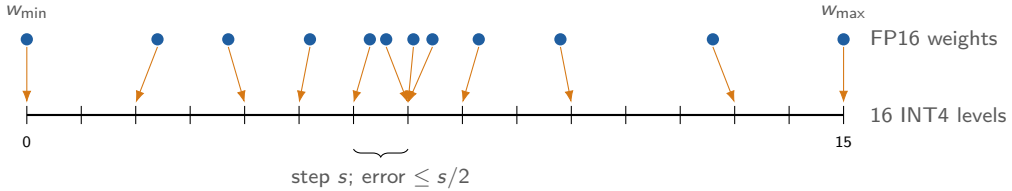
1. In FP16?
   $7 \times 2 = 14$ GB $>$ 12 GB: it does not fit. Even if it did, $60/14 \approx 4$ tok/s.
2. In INT4?
   3.5 GB fits, but decode is at most $60/3.5 \approx 17$ tok/s, still short of 20.
3. What would you change?
   A smaller model (3B INT4: $\leq$40 tok/s), fewer weights read per token (MoE, Sec. 3), or run it somewhere else (Sec. 5–6).

Roadmap

1. The device's budget: memory, bandwidth, energy
2. Quantization: fewer bits per weight
3. Mixture of Experts: capacity without compute
4. The KV cache: memory that grows with the conversation
5. Split and collaborative inference
6. Model routing: choosing the model per request
7. Wrap-up

Quantization in one picture

$$q = \mathrm{clamp}\Big(\mathrm{round}\big(\tfrac{w}{s}\big) + z,\ 0,\ 2^b - 1\Big), \qquad \hat{w} = s\,(q - z), \qquad s = \frac{w_{\max} - w_{\min}}{2^b - 1}$$



Granularity: one scale per tensor → per output channel → per group of 32–128 weights. Finer groups are more accurate but cost metadata. An FP16 scale and zero point per 128 weights add 0.25 bit, so "INT4, g128" ≈ **4.25 bits/weight**.

What to quantize, and what it buys

Scheme	W / A bits	Main win	Where
W8A8	8 / 8	faster **prefill** (integer matmuls); 2× smaller	NPUs, servers
W4A16 (weight-only)	4 / 16	faster **decode** (4× fewer bytes); 4× smaller	phones, laptops
KV-cache quant.	KV: 2–4	longer context in the same memory	Section 4

Rules of thumb for LLMs

- 8-bit weights: close to lossless, *if* outliers are handled (next frame).
- 4-bit weights with a good post-training method (GPTQ, AWQ): small accuracy loss.
- 3 bits and below: usually needs **quantization-aware training** (QAT), i.e., training with quantization in the loop. MobileMoE ships with INT4 QAT.

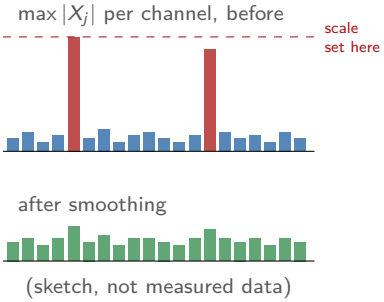
The catch: outliers

LLM.int8() (NeurIPS'22): a few hidden dimensions carry huge activations ("emergent outliers"). A scale sized for them leaves almost no resolution for the rest. *Fix*: keep those dimensions in FP16; >99.9% of values still multiply in INT8.

SmoothQuant (ICML'23): weights are easy to quantize and activations are hard, so move the difficulty with an exact rewrite:

$$Y = \big(X\,\mathrm{diag}(s)^{-1}\big)\big(\mathrm{diag}(s)\,W\big), \quad s_j = \frac{\max|X_j|^{\alpha}}{\max|W_j|^{1-\alpha}}$$

$\Rightarrow$ W8A8 with up to 1.56× speedup and 2× less memory.



T. Dettmers et al., LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, NeurIPS 2022 · G. Xiao et al., SmoothQuant, ICML 2023

Getting to 4 bits: GPTQ and AWQ

GPTQ (ICLR'23)

- Quantize one weight column at a time.
- After each column, update the not-yet-quantized columns to cancel its error, using second-order (Hessian) information from a small calibration set.
- A 175B model goes to 3–4 bits in $\approx$4 GPU hours with negligible accuracy loss.

AWQ (MLSys'24)

- Not all weights matter equally. About 1% of *salient* channels, found from *activation* magnitudes, dominate the error.
- Protect them by scaling those channels before quantization. No backpropagation, no reconstruction.
- With the TinyChat runtime: 4$\times$ smaller models and 3–4$\times$ faster on edge platforms.

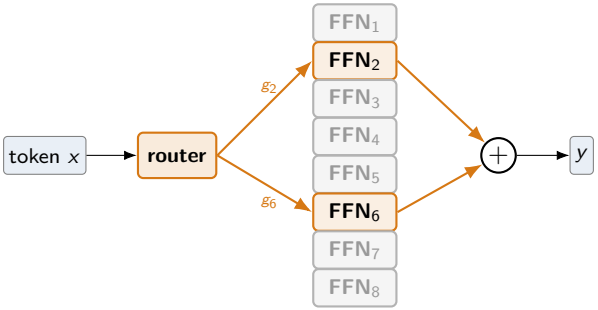
Both are **post-training** quantization (PTQ): minutes to hours on one GPU, no retraining. On 11/4, **D2MoE** goes further and picks a precision *per expert* at run time.

E. Frantar et al., GPTQ, ICLR 2023 · J. Lin et al., AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration, MLSys 2024

Roadmap

1. The device's budget: memory, bandwidth, energy
2. Quantization: fewer bits per weight
3. Mixture of Experts: capacity without compute
4. The KV cache: memory that grows with the conversation
5. Split and collaborative inference
6. Model routing: choosing the model per request
7. Wrap-up

Mixture of Experts: a router inside the layer



8 experts stored, top-2 used

$$g(x) = \mathrm{softmax}(W_r\, x)$$

$$y = \sum_{i \in \mathrm{TopK}(g(x),\, k)} g_i(x)\, \mathrm{FFN}_i(x)$$

- Sparsely gated MoE: Shazeer et al. (ICLR'17).
- Switch Transformer routes each token to k=1 expert (JMLR'22).
- Each token *touches* only k experts, but all E experts must be *stored*.

N. Shazeer et al., Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer, ICLR 2017 · W. Fedus, B. Zoph, N. Shazeer, Switch Transformers, JMLR 2022

MoE trades memory for compute

Model	Total params	Active per token	Design
Mixtral 8x7B	47B	13B	top-2 of 8 experts
DeepSeek-V3	671B	37B	fine-grained + shared experts
MobileMoE family	1.3–5.3B	0.3–0.9B	fine-grained + shared, for phones

- Prefill FLOPs $\propto$ **active** params.
- Bytes read per decode step $\propto$ **active** params, *if every expert sits in DRAM*.
- Memory footprint $\propto$ **total** params.

- **Data center**: memory is pooled across GPUs and compute is the bill. MoE wins, so frontier models are MoEs.
- **Phone**: memory is the scarcest resource. Does MoE still pay off?
- MobileMoE (today): yes, at moderate sparsity. At equal INT4 weight memory it decodes **2.2–3.4$\times$ faster** than a dense model.

A. Q. Jiang et al., Mixtral of Experts, arXiv:2401.04088, 2024 · DeepSeek-AI, DeepSeek-V3 Technical Report, arXiv:2412.19437, 2024 · Y. Chen et al., MobileMoE, arXiv:2605.27358, 2026

Fine-grained and shared experts

(a) conventional: 8 experts, top-2



(b) fine-grained + shared: 1 shared, 32 small, top-8



same active compute, but $\binom{8}{2} = 28$ vs. $\binom{32}{8} \approx 10^7$ possible expert combinations

DeepSeekMoE (ACL'24)

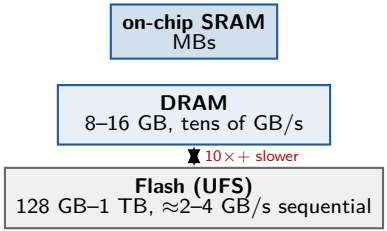
- Split each expert into m smaller ones and activate $m\times$ as many: sharper specialization.
- Isolate a few *shared* experts that every token uses, so routed experts do not duplicate common knowledge.

MobileMoE's on-device sweet spot

- Moderate sparsity, fine-grained experts, one shared expert, e.g., **60 routed + 1 shared, top-4**. Found with a scaling law fitted under phone limits.

D. Dai et al., DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models, ACL 2024 · Y. Chen et al., MobileMoE, arXiv:2605.27358, 2026

When the experts do not fit: offload to flash



small random reads from flash are slower still

- **LLM in a flash** (ACL'24): keep weights on flash and load only what is needed. It reuses recently loaded neurons ("windowing") and reads larger contiguous chunks ("row–column bundling"). Models up to 2× the DRAM size run 4–5× (CPU) and 20–25× (GPU) faster than with naive loading.
- **The MoE twist**: the router decides which expert it needs only at the last moment. So cache the hot experts in DRAM, and predict and prefetch the rest.
- **D2MoE** (MobiCom'25, on 11/4): route to an expert *and* a precision, and overlap expert loading with compute.

K. Alizadeh et al., LLM in a Flash: Efficient Large Language Model Inference with Limited Memory, ACL 2024 · H. Wang et al., D2MoE, ACM MobiCom 2025

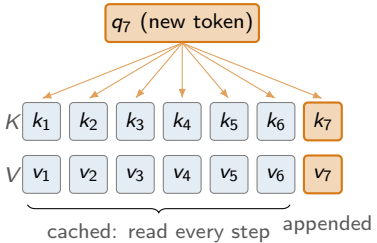
Roadmap

1. The device's budget: memory, bandwidth, energy
2. Quantization: fewer bits per weight
3. Mixture of Experts: capacity without compute
4. The KV cache: memory that grows with the conversation
5. Split and collaborative inference
6. Model routing: choosing the model per request
7. Wrap-up

Why decoding keeps a KV cache

$$o_t = \mathrm{softmax}\Big(\frac{q_t\, K_{1:t}^{\top}}{\sqrt{d}}\Big)\, V_{1:t}$$

- The keys and values of past tokens never change, so store them and append one row per step.
- Without the cache, every step recomputes all t tokens: $O(t^2)$ total work. With it, each step does $O(t)$ work.
- The price: memory grows *linearly* with context, and every decode step reads the whole cache. That is bandwidth again.



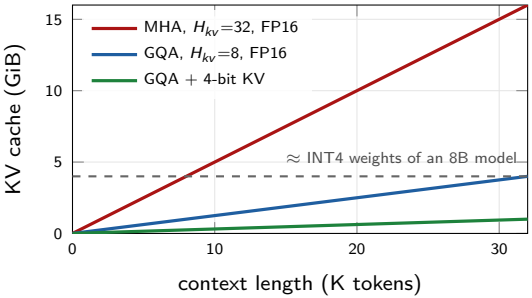
How big is the KV cache?

$$\text{KV bytes} \;=\; 2 \times L \times H_{kv} \times d_{\text{head}} \times n_{\text{ctx}} \times \text{bytes/elem}$$

Llama-2-7B: L=32, H_{kv}=32, d_{head}=128, FP16
$\Rightarrow$ 512 KiB/token; 2 GiB at 4K tokens.

Llama-3-8B: grouped-query attention, H_{kv}=8
$\Rightarrow$ 128 KiB/token; 1 GiB at 8K, **16 GiB at 128K**.

At long context, the cache rivals or exceeds the weights.



Lines use an 8B-class shape (L=32, d_{head}=128). Configurations from H. Touvron et al., Llama 2, arXiv:2307.09288, 2023 · Llama Team (Meta), The Llama 3 Herd of Models, arXiv:2407.21783, 2024

Four knobs to shrink the KV cache

1. Fewer KV heads — H_{kv}

MQA shares one K/V head across all query heads. **GQA** (EMNLP'23) shares one per group: near-MHA quality at near-MQA speed. Llama-3-8B: 32 query heads, 8 KV heads $\Rightarrow$ 4$\times$ smaller.

2. Fewer tokens — n_{ctx}

H2O (NeurIPS'23) keeps "heavy-hitter" tokens plus recent ones. **StreamingLLM** (ICLR'24) keeps a few "attention-sink" tokens plus a sliding window.

3. Fewer bits — bytes/elem

KIVI (ICML'24): 2-bit KV cache, keys per channel and values per token. Almost the same quality with 2.6$\times$ less peak memory.

4. Better placement — where it lives

PagedAttention (SOSP'23) pages the cache like an OS to stop fragmentation. **KVSwap** (MobiSys'26, on 11/4) keeps it on flash and prefetches what is needed next.

The KV cache is your conversation in another form, so it is a privacy target too (KV-Cloak, 11/23).

Shazeer, arXiv 2019 (MQA) · Ainslie et al., EMNLP 2023 · Z. Zhang et al., NeurIPS 2023 · G. Xiao et al., ICLR 2024 · Z. Liu et al., ICML 2024 · Kwon et al., ACM SOSP 2023 · H. Zhang et al., ACM MobiSys 2026. Full titles in the References.

Exercise: does it fit in 4 GB?

A **3B** GQA model: L=28, H_{kv}=8, d_{head}=128. App memory budget: **4 GB**. Phone bandwidth: ≈**60 GB/s**.

1. Weights at INT4 (g128, ≈4.25 bits)?
2. KV cache per token in FP16? For 8K tokens? For 32K tokens?
3. Does a 32K-token chat fit?
4. Decode upper bound at 8K context?

Exercise: does it fit in 4 GB?

A **3B** GQA model: L=28, H_{kv}=8, d_{head}=128. App memory budget: **4 GB**. Phone bandwidth: ≈**60 GB/s**.

1. Weights at INT4 (g128, ≈4.25 bits)?
   $3 \times 10^9 \times 4.25/8 \approx$ **1.6 GB**
2. KV cache per token in FP16? For 8K tokens? For 32K tokens?
3. Does a 32K-token chat fit?
4. Decode upper bound at 8K context?

Exercise: does it fit in 4 GB?

A **3B** GQA model: L=28, H_{kv}=8, d_{head}=128. App memory budget: **4 GB**. Phone bandwidth: ≈**60 GB/s**.

1. Weights at INT4 (g128, ≈4.25 bits)?
   $3 \times 10^9 \times 4.25/8 \approx$ **1.6 GB**
2. KV cache per token in FP16? For 8K tokens? For 32K tokens?
   $2 \cdot 28 \cdot 8 \cdot 128 \cdot 2$ B = **112 KiB**/token, so ≈0.9 GB at 8K and ≈3.8 GB at 32K
3. Does a 32K-token chat fit?
4. Decode upper bound at 8K context?

Exercise: does it fit in 4 GB?

A **3B** GQA model: L=28, H_{kv}=8, d_{head}=128. App memory budget: **4 GB**. Phone bandwidth: ≈**60 GB/s**.

1. Weights at INT4 (g128, ≈4.25 bits)?
   $3 \times 10^9 \times 4.25/8 \approx$ **1.6 GB**
2. KV cache per token in FP16? For 8K tokens? For 32K tokens?
   $2 \cdot 28 \cdot 8 \cdot 128 \cdot 2$ B = **112 KiB**/token, so ≈0.9 GB at 8K and ≈3.8 GB at 32K
3. Does a 32K-token chat fit?
   $1.6 + 3.8 > 4$ GB ×. With a 4-bit KV cache (≈0.95 GB) ✓, or evict or offload tokens.
4. Decode upper bound at 8K context?

Exercise: does it fit in 4 GB?

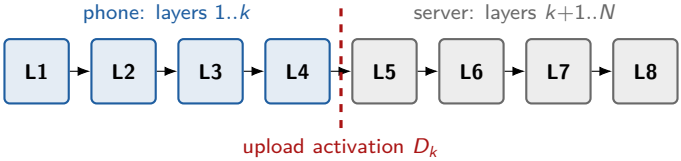
A **3B** GQA model: L=28, H_{kv}=8, d_{head}=128. App memory budget: **4 GB**. Phone bandwidth: ≈**60 GB/s**.

1. Weights at INT4 (g128, ≈4.25 bits)?
   $3 \times 10^9 \times 4.25/8 \approx$ **1.6 GB**
2. KV cache per token in FP16? For 8K tokens? For 32K tokens?
   $2 \cdot 28 \cdot 8 \cdot 128 \cdot 2$ B = **112 KiB**/token, so ≈0.9 GB at 8K and ≈3.8 GB at 32K
3. Does a 32K-token chat fit?
   $1.6 + 3.8 > 4$ GB ×. With a 4-bit KV cache (≈0.95 GB) ✓, or evict or offload tokens.
4. Decode upper bound at 8K context?
   Each token reads $\approx 1.6 + 0.9 = 2.5$ GB, so at most $60/2.5 \approx$ **24 tok/s**.

Roadmap

1. The device's budget: memory, bandwidth, energy
2. Quantization: fewer bits per weight
3. Mixture of Experts: capacity without compute
4. The KV cache: memory that grows with the conversation
5. Split and collaborative inference
6. Model routing: choosing the model per request
7. Wrap-up

Before LLMs: offload, or split the network?

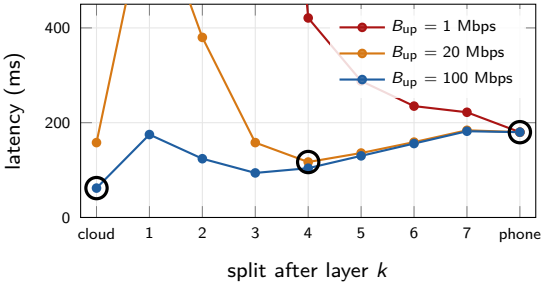


- **MAUI** (MobiSys'10): decide at run time which *methods* to execute remotely, to save energy under current connectivity. Example: face recognition at an order of magnitude less energy.
- **Neurosurgeon** (ASPLOS'17): split a DNN at a *layer* boundary. Profile per-layer latency and energy, then pick the cut that minimizes end-to-end latency or phone energy.
- Key fact: activation size varies widely across layers. Early CNN layers can be larger than the input image; late layers are tiny.

E. Cuervo et al., MAUI: Making Smartphones Last Longer with Code Offload, ACM MobiSys 2010 · Y. Kang et al., Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge, ACM ASPLOS 2017

Choosing the split point: the network decides

$$k^\star = \arg\min_k \Big[\underbrace{\textstyle\sum_{i\le k} t_i^{\text{dev}}}_{\text{phone}} + \underbrace{\frac{D_k}{B_{\text{up}}} + \text{RTT}}_{\text{network}} + \underbrace{\textstyle\sum_{i>k} t_i^{\text{srv}}}_{\text{server}} \Big] \qquad (k{=}N\text{: no network term})$$

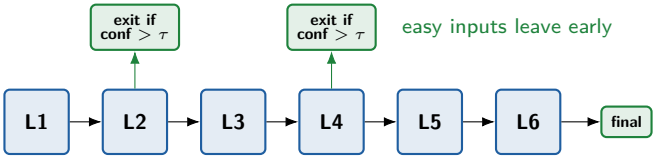


- 1 Mbps: keep everything on the phone.
- 20 Mbps: cut after layer 4.
- 100 Mbps: just send the input.

The best placement moves with the uplink.
5G uplink is much weaker than downlink and fluctuates, so placement must be **network-aware**.

Illustrative 8-layer profile: phone 180 ms in total, server 10× faster, RTT 20 ms, activations from 1.6 MB down to 5 KB. Circles mark each curve's optimum.

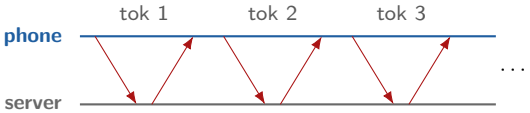
Early exits: stop as soon as you are confident



- **BranchyNet** (ICPR'16): attach classifiers to intermediate layers so that easy inputs exit early.
- **SPINN** (MobiCom'20): combine early exits with device–cloud splitting. A run-time scheduler co-tunes the split point and the exit policy to the network, giving up to 2× throughput over prior collaborative inference.
- The LLM-era analogue is *elastic* models. **ElastiLM** (MobiCom'25, on 11/4) shrinks the model and the prompt per request to meet each app's deadline.

S. Teerapittayanon, B. McDanel, H. T. Kung, BranchyNet, ICPR 2016 · S. Laskaridis et al., SPINN, ACM MobiCom 2020 · W. Yin et al., Elastic On-Device LLM Service, ACM MobiCom 2025

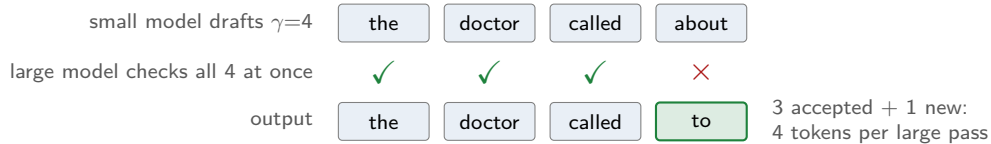
Why splitting an LLM is different



- A CNN crosses the network **once per input**. An LLM split in the middle crosses it **once per output token**.
- 200-token answer $\times$ 30 ms RTT $=$ **6 s** of pure waiting.
- The payload is tiny: one hidden vector is 4096×2 B $= 8$ KB. The split is *latency*-bound, not bandwidth-bound.

For LLMs, the goal shifts from *fewer bytes* to **fewer round trips**.

Speculative decoding: draft cheaply, verify in one pass



- Leviathan et al. (ICML'23): the output distribution is **identical** to the large model's; 2–3× faster on T5-XXL.
- With acceptance rate α, the expected tokens per large-model pass are $\frac{1-\alpha^{\gamma+1}}{1-\alpha}$. For α=0.8, γ=4: ≈**3.4 tokens**.
- **Edge twist**: draft on the phone, verify on the server: one round trip per ∼3 tokens instead of per token.

Y. Leviathan, M. Kalman, Y. Matias, Fast Inference from Transformers via Speculative Decoding, ICML 2023

Four ways to collaborate

Split by...	Who does what	Example
layers	phone runs the first k layers, the server the rest	Neurosurgeon (ASPLOS'17)
role	phone drafts tokens, the server verifies them	speculative decoding (ICML'23)
time	server writes the hard opening, then hands off once	SAIL (MobiSys'26)
device	one model sliced across home devices as a pipeline	Confidant (MobiCom'25)

And the edge itself is not a guarantee

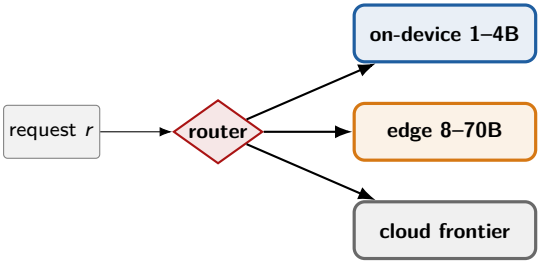
SMEC (NSDI'26): on commercial 5G edge deployments, both the RAN scheduler and the edge server are shared and blind to deadlines. SMEC puts a deadline-aware manager at each, using only standard 5G signals.

G. Park et al., SAIL, ACM MobiSys 2026 · Y. Chen et al., Confidant, ACM MobiCom 2025 · X. Zhang et al., Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC, USENIX NSDI 2026

Roadmap

1. The device's budget: memory, bandwidth, energy
2. Quantization: fewer bits per weight
3. Mixture of Experts: capacity without compute
4. The KV cache: memory that grows with the conversation
5. Split and collaborative inference
6. Model routing: choosing the model per request
7. Wrap-up

Model routing: pick the model per request



top → bottom: higher quality,
but more cost, latency, and data exposure

$$m^\star(r) = \arg\max_m \ \widehat{Q}(m, r) - \lambda\,\mathrm{Cost}(m, r)$$

s.t. deadline and privacy constraints

Routing, twice

Network routing picks a path per flow from link costs.

Model routing picks a model per request from *predicted* quality and cost.

The hard part: predicting quality *before* paying for the big model.

How routers predict quality

Cascade: try small first

AutoMix (NeurIPS'24): the small model answers and checks itself (few-shot self-verification). A POMDP router escalates when it is unsure.

$\Rightarrow$ over 50% lower cost at comparable performance.

– hard queries pay twice.

Predict difficulty upfront

Hybrid LLM (ICLR'24): a router predicts the quality gap between the small and large model for each query. A threshold, tunable at test time, trades quality for cost.

$\Rightarrow$ up to 40% fewer calls to the large model with no quality drop.

Learn from preferences

RouteLLM (ICLR'25): routers trained on human preference data, with data augmentation.

$\Rightarrow$ over 2$\times$ lower cost without losing quality, and the routers transfer to unseen model pairs.

All three were built around cloud cost. At the edge, "cost" also includes battery, RTT, and privacy.

P. Aggarwal et al., AutoMix: Automatically Mixing Language Models, NeurIPS 2024 · D. Ding et al., Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing, ICLR 2024 · I. Ong et al., RouteLLM: Learning to Route LLMs from Preference Data, ICLR 2025

Routing at the edge sees more than the prompt

Request	Binding constraint	Route
"Suggest the next word"	deadline under 50 ms; tiny task	**device**
"Summarize my doctor's voicemail"	privacy	**device** (or a trusted cloud)
"Translate live on my AR glasses"	∼100 ms, needs a bigger model	**edge**
"Plan a 5-day trip with web search"	hard, not urgent	**cloud**

Signals an edge router can use: query difficulty · deadline · privacy label · RTT and uplink · battery and temperature · price

Routing happens at three granularities

token → expert	request → model	request → place
MoE router (Sec. 3)	LLM router (this section)	device / edge / cloud (Sec. 5)

Roadmap

1. The device's budget: memory, bandwidth, energy
2. Quantization: fewer bits per weight
3. Mixture of Experts: capacity without compute
4. The KV cache: memory that grows with the conversation
5. Split and collaborative inference
6. Model routing: choosing the model per request
7. Wrap-up

Takeaways

1. On a phone, **memory capacity and bandwidth**, not FLOPs, set the limits: decode speed $\approx$ bandwidth / bytes per token.
2. **Quantization** cuts bytes, which improves memory, speed, and energy at once. Outliers are the enemy (LLM.int8(), SmoothQuant, GPTQ, AWQ).
3. **MoE** buys capacity with memory. On phones it pays only if the experts are resident, or are predicted and prefetched well.
4. The **KV cache** grows with context and can rival the weights. Shrink heads, tokens, or bits, or move it.
5. **Splitting** suits one-pass models. For LLMs, cut *round trips*: speculative decoding, a single handoff.
6. **Routing** decides per request: device, edge, or cloud, using difficulty, deadline, privacy, link, and battery.

Coming up

Now: paper presentation (15 min + 5 min Q&A)

MobileMoE: Scaling On-Device Mixture of Experts (Meta, arXiv, May 2026), presented by **Lex Schneier**.
Listen for: what is held fixed in their scaling law, and what it assumes about memory.

11/4 · Practicum 8 (Edge AI I)

ElastiLM (MobiCom'25): one LLM, many deadlines
D2MoE (MobiCom'25): experts that do not fit
KVSwap (MobiSys'26): KV cache on flash
HW 4 due; final project released.

11/9 · Practicum 9 (Edge AI II)

SAIL (MobiSys'26): one server-to-mobile handoff
SMEC (NSDI'26): deadline-aware 5G edge
Confidant (MobiCom'25): training across devices

For every paper, ask: which term of $T(r, p)$, or which constraint, does it attack?

References (1/2)

On-device models and measurement

- Apple. Apple Intelligence Foundation Language Models. arXiv:2407.21075, 2024.
- Z. Liu et al. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. *ICML*, 2024.
- Y. Chen et al. MobileMoE: Scaling On-Device Mixture of Experts. arXiv:2605.27358, 2026.
- S. Laskaridis, K. Katevas, L. Minto, H. Haddadi. MELTing Point: Mobile Evaluation of Language Transformers. *ACM MobiCom*, 2024.
- D. Xu et al. Fast On-device LLM Inference with NPUs. *ACM ASPLOS*, 2025.
- Y. Zhong et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. *USENIX OSDI*, 2024.
- S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4), 2009.
- M. Horowitz. Computing's Energy Problem (and What We Can Do About It). *IEEE ISSCC*, 2014.

Quantization

- T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *NeurIPS*, 2022.
- G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *ICML*, 2023.
- E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *ICLR*, 2023.
- J. Lin et al. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *MLSys*, 2024.

Mixture of Experts

- N. Shazeer et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *ICLR*, 2017.
- W. Fedus, B. Zoph, N. Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *JMLR*, 23, 2022.
- D. Dai et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *ACL*, 2024.
- A. Q. Jiang et al. Mixtral of Experts. arXiv:2401.04088, 2024.
- DeepSeek-AI. DeepSeek-V3 Technical Report. arXiv:2412.19437, 2024.
- K. Alizadeh et al. LLM in a Flash: Efficient Large Language Model Inference with Limited Memory. *ACL*, 2024.
- H. Wang, Q. Zhou, Z. Hong, S. Guo. D2MoE: Dual Routing and Dynamic Scheduling for Efficient On-Device MoE-based LLM Serving. *ACM MobiCom*, 2025.

References (2/2)

KV cache

- H. Touvron et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288, 2023.
- Llama Team, Meta. The Llama 3 Herd of Models. arXiv:2407.21783, 2024.
- N. Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150, 2019.
- J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *EMNLP*, 2023.
- Z. Zhang et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS*, 2023.
- G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. Efficient Streaming Language Models with Attention Sinks. *ICLR*, 2024.
- Z. Liu et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *ICML*, 2024.
- W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *ACM SOSP*, 2023.
- H. Zhang, C. Xia, Z. Wang. KVSwap: Disk-aware KV Cache Offloading for Long-Context On-device Inference. *ACM MobiSys*, 2026.

Split and collaborative inference

- E. Cuervo et al. MAUI: Making Smartphones Last Longer with Code Offload. *ACM MobiSys*, 2010.
- Y. Kang et al. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. *ACM ASPLOS*, 2017.
- S. Teerapittayanon, B. McDanel, H. T. Kung. BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks. *ICPR*, 2016.
- S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, N. D. Lane. SPINN: Synergistic Progressive Inference of Neural Networks over Device and Cloud. *ACM MobiCom*, 2020.
- Y. Leviathan, M. Kalman, Y. Matias. Fast Inference from Transformers via Speculative Decoding. *ICML*, 2023.
- W. Yin et al. Elastic On-Device LLM Service. *ACM MobiCom*, 2025.
- G. Park et al. SAIL: Redesigning Collaborative Language Inference with a Single Server-to-Mobile Handoff. *ACM MobiSys*, 2026.
- X. Zhang et al. Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC. *USENIX NSDI*, 2026.
- Y. Chen et al. Confidant: Customizing Transformer-based LLMs via Collaborative Training on Mobile Devices. *ACM MobiCom*, 2025.

Model routing

- P. Aggarwal et al. AutoMix: Automatically Mixing Language Models. *NeurIPS*, 2024.
- D. Ding et al. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. *ICLR*, 2024.
- I. Ong et al. RouteLLM: Learning to Route LLMs from Preference Data. *ICLR*, 2025.